Chemistry AI Research

Tanishq Malhotra

Norm Tubman

October 5th, 2025

Overview

The project combines Molecular Dynamics simulation and Neural Networks modelling to study and predict the potential energy of atomic systems. The project is divided into two parts. The first part involves simulating the motion of two particles connected by a spring based on Hooke's law, using the Euler Method. This process generates one million data points representing the particles' positions and corresponding energies over time. The second part uses the generated data to train a **Neural Network in the Behler and Parinello Architecture** which is used to approximate potential energy surfaces with DFT level accuracy with much less computational cost.

Project Goals

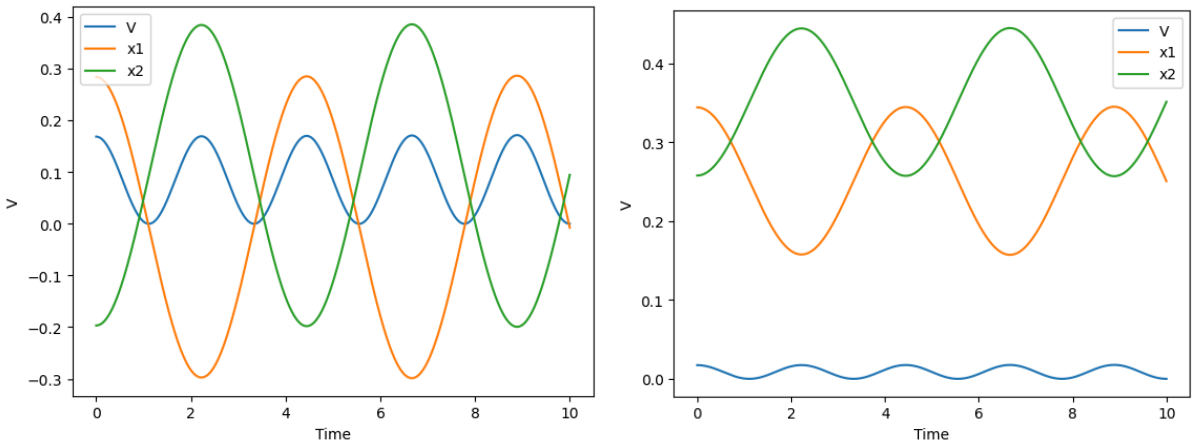
The project aims to simulate particle interactions using basic MD to generate realistic position and energy data for simple systems. A Behler style neural network is developed and trained to predict potential energies from these atomic positions. This approach bridges simulation and machine learning by demonstrating how neural networks can efficiently replicate quantum-accurate potential energy surfaces (PES). Additionally, the model ensures scalability and maintains essential physical symmetries such as translation, rotation, and permutation invariance, making it both accurate and generalizable

Understanding the Generalized Neural-Network Representation Paper

Behler and Parrinello's 2007 research proposed a generalized neural network architecture to represent potential energy surfaces (PES) with Density Functional Theory level precision but at a

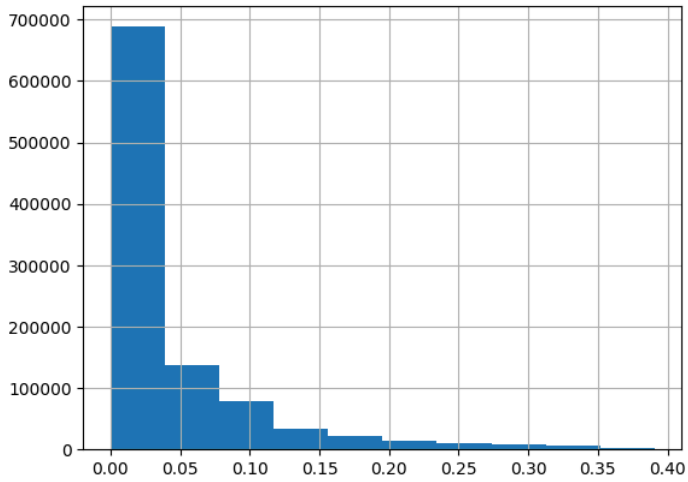
fraction of the computational cost. DFT, while highly accurate, is extremely expensive with simulations limited to only a few thousand atoms and timescales below 50 picoseconds. To overcome this, Behler and Parrinello expressed the total energy of a molecular system as a summation of atomic energy contributions, where each atomic term is predicted by a sub-network trained to recognize its local environment. Each sub-network shares the same weights, ensuring that identical atoms contribute equally to the total energy, regardless of order.

A central part of the architecture includes the use of **symmetry functions** to encode an atom's local surroundings. Each atom's environment is represented through a **cutoff, radial and angular symmetry functions** that depend on distances and angles between neighboring atoms within a cutoff radius of **6 Å**. These functions ensure that the network's inputs remain invariant under rotation, translation, and permutation — preserving the physical symmetry of atomic systems. Radial symmetry functions were constructed using exponential decay terms centered at different distances to capture near and mid-range interactions, while angular terms incorporated cosine functions to capture bond angles and three-body effects. In my neural network implementation, the same principle was applied: atomic positions were converted into numerical representations that depend only on interatomic distances.



Data Generation

The Molecular Dynamics (MD) simulation forms the foundation for training data. In my setup, two particles connected by a virtual spring were simulated over **1,000,000 time steps** using the **Euler integration method** with a **timestep of 0.01 picoseconds**. The spring constant kkk was set to 1 N/m, and both particles were initialized with unit mass and small displacements to produce harmonic oscillations. The resulting data includes **time, position (x_1, x_2), velocity, and potential energy (V)**. The graph of this simulation shows periodic oscillations in x_1 and x_2, while potential energy remains bounded below 0.05 eV, indicating stable energy exchange between the two masses.



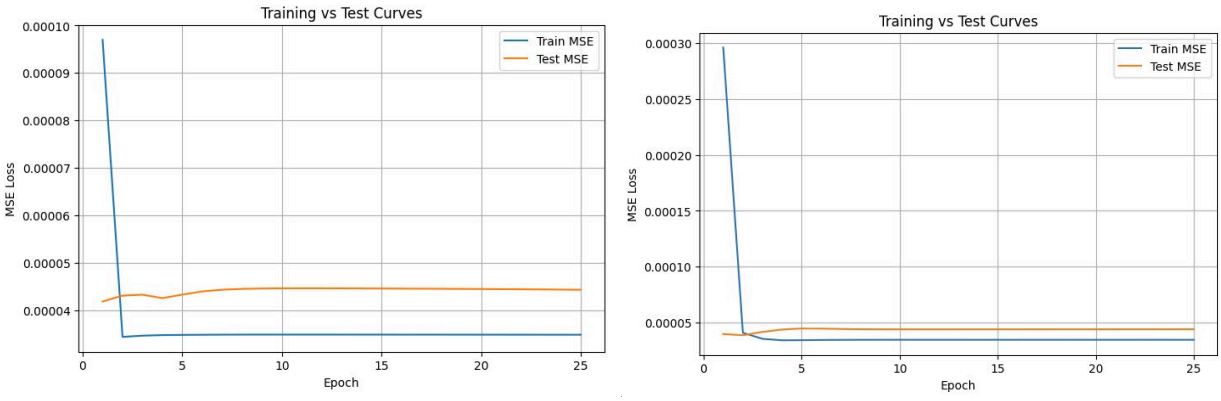

```python
def simulate(x, v, k, d, V_min, steps):
    """
    x: initial position of particles
    v: initial velocity of particles
    k: hookes constant
    d: equilibrium distance between particles
    V_min: min energy of system

    returns
    xs_hist: position of all particles at all timesteps
        [positions_of_n_particles_at_time_0, positions_of_n_particles_at_time_1, ...]
    V_hist: energy of system at all timesteps
        [energy_at_time_0, energy_at_time_1, ...]
    """
    xs_hist = [x[:]]
    vs_hist = [v[:]]
    V_hist =  [eqn5(x[:], k, d, V_min)]

    for _ in range(steps):
        f = eqn6(x, k, d)
        x = eqn11(x, v, h)
        v = eqn12(m, v, h, f)
        V = eqn5(x, k, d, V_min)
        print('x', x, 'k', k, 'd', d, 'V_min', V_min, 'V',V)
        xs_hist.append(x[:])
        vs_hist.append(v[:])
        V_hist.append(V)

    return xs_hist, V_hist
```

```python
total_time=10
h=0.001 # timestep diff
k=1.0
m=1.0 # mass
d=0.1
V_min = 0.0
steps = int(total_time / h)
filepath = "./simulation_data_2_particles.csv"
num_simulations = 100
random.seed(0)
```

Testing and Training

The neural network was trained using the dataset generated from the Molecular Dynamics simulation, consisting of approximately 1,000,000 samples of particle positions and corresponding potential energies. However, to reduce the amount of time to run the Neural Network, the neural network is run on 10,000 samples.The data is divided into 80% for training and 20% for testing to ensure proper model validation. Each training sample contained the particle positions as input and the system's potential energy V as the target output. During

training, the model optimized its weights using backpropagation and the Mean Squared Error (MSE) loss function, which penalizes deviations between the predicted and true energy values. The training process was conducted over multiple epochs (typically 100–200), with the Adam optimizer used to improve convergence speed and stability. Two runs of the neural network are below.



Infrastructure

The Molecular Dynamics simulation and data preprocessing were run locally on a MacBook with Apple Silicon, which efficiently handled the generation of over one million data points. The neural network was then implemented using PyTorch and trained entirely on Google Collab, using the T4 GPU runtime to accelerate matrix operations and backpropagation. This setup significantly reduced training time and allowed for quick experimentation with different learning rates, batch sizes, and network architectures.

Research Papers I Read

During my 3 months research project. I read 3 Research Papers. These consisted of the The Open Molecules 2025 (OMol25), Generalized Neural-Network Representation of

High-Dimensional Potential-Energy Surfaces and the A Hands-On Introduction to Molecular Dynamics. The main focus of the project being the latter two papers.

The OMOL 2025 Paper discussed the creation of the Meta Fair's massive dataset containing 100 million density functional theory calculations to address the lack of large scale molecular data. The dataset was tested on the seven benchmark cases to assess the OMOL mode's accuracy and highlight areas of improvement. The main goal of the paper was to provide precise real world ready data for drug discovery and a benchmark for future molecular ML research.

The Generalized Neural-Network Representation introduces a scalable model designed to overcome the symmetry and scalability limitations of traditional neural networks. Each atom's energy is predicted by a sub-network sharing the same weights, ensuring consistency across different systems. This structure allows the total energy to be expressed as the sum of atomic contributions, make the model efficient for larger molecular systems.

The paper uses symmetry functions to predict energies and forces across different phases that are invariant to rotation and permutation, ensuring that predictions remain accurate regardless of molecular orientation or atom order. Radial and Angular Symmetry functions capture both distance and angle information of each atom, allowing Neural Networks to replicate DFT level accuracy with less computational effort.

The Introduction to Molecular Dynamics highlights the gap between theory and laboratory research. The MD simulations use Newton's laws of motions to simulate particle interaction, collision and how they evolve in a system. By following these trajectories, scientists

can calculate properties such as temperature and pressure. This makes it an essential tool for linking theoretical models with experimental results.

In MD simulations, forces between particles are derived from potential energy functions, allowing modelling of atomic interactions. The accuracy of these simulations depends on the chosen potential and the integration method used to update positions and velocities. Numerical methods such as Verlet and Euler provide stable and precise results over long periods of times, minimising cumulative errors. Overall MD serves as a bridge between theory and experiment as it offers detailed insights into fundamental processes into chemical systems.

Difficulties

The main challenge was translating theoretical research into functional code. Implementing the neural network required converting symmetry functions and atomic summations into working PyTorch operations while ensuring rotational and permutational invariance. Training on over a million samples also introduced issues with speed and stability, such as slow convergence and gradient explosions. Tuning of the learning rate, activation functions, and optimizers like Adam was essential to achieve stable results.

References

1. Behler, J., & Parrinello, M. (2007). *Generalized neural-network representation of high-dimensional potential-energy surfaces. Physical Review Letters, 98*(14), 146401. https://doi.org/10.1103/PhysRevLett.98.146401

2. Meta FAIR. (2025). *Open Molecular (OMol25) Dataset: 100 Million DFT Calculations for Machine Learning Potentials*
3. Lamberti, V. E., Fosdick, L. D., & Jessup, E. R. (2002). A Hands-On Introduction to Molecular Dynamics. Journal of Chemical Education.